PROJECT CONTEXT STATE — TabGis (Para uso com IA)

Leitura obrigatória antes de qualquer sessão de desenvolvimento. Este ficheiro é a fonte de verdade do estado operacional do TabGis. Actualize-o sempre que um componente mudar de estado.

Ambiente de execução

Componente	Detalhe
Servidor de execução	`homeserver` — Linux (Ubuntu), acesso via SSH
Desenvolvimento	Cursor IDE com extensão **Remote SSH** (qualquer notebook → `ssh homeserver`)
Repositório	`/home/wesley/projects/TabGis` no homeserver
Orquestração	Docker Compose em `infra/docker-compose.yml`

Atenção: Existe também um `docker-compose.yml` na raiz do repositório (composição experimental com pgRouting/kartoza). **Não** é o alvo dos runbooks actuais — usar sempre `infra/docker-compose.yml`.

Versionamento e código-fonte

Artefacto	Local / URL
Repositório Git (remoto)	`https://github.com/wesleyspo/TabGis.git`
Cópia de trabalho (homeserver)	`/home/wesley/projects/TabGis`
Release V0 (marco base)	**v0.1.0** — *tag* anotada Git
Ficheiro de versão	`VERSION` na raiz (ex.: `0.1.0`)

O código da aplicação (frontend, `infra/`, `backend/`, SQL, documentação) está versionado neste repositório Git. A linha temporal oficial é o histórico de *commits* em `main`; marcos estáveis assinalam-se com *tags* `vMAJOR.MINOR.PATCH` (SemVer). Dados pesados e artefactos gerados localmente **não** entram no Git — ver `.gitignore` (`data/raw/`, `*.mbtiles`, `*.mbtiles.tmp`, etc.).

Comandos úteis no homeserver:

```
git describe --tags --always      # tag actual ou commit abreviado
git tag -l 'v*'                   # listar releases
git show v0.1.0 --no-patch        # nota da tag V0
```

Para publicar alterações no remoto (após `git commit`):

```
git push origin main
git push origin v0.1.0            # ou: git push origin --tags
```

Stack Docker — estado actual (todos healthy)

Container	Imagem	Porta host	Função
infra-db-1	postgis/postgis:15-3.3	**5433**→5432 (host→container)	PostgreSQL + PostGIS — **evitar mapear 5432:5432** no host se já existir outro PostgreSQL local
infra-geoserver-1	oscarfonts/geoserver:latest	**8080**	WMS / WFS / SLD
infra-tileserver-1	maptiler/tileserver-gl:latest	**8081**	MBTiles raster (satélite local)
infra-web-1	nginx:alpine	**8088**	Frontend + proxy /geoserver, /tiles, /api

Container	Imagem	Porta host	Função
infra-api-1	build backend/ (Node)	**3000**	API REST (via :8088/api/): GET /api/translations/:lang; GET/POST/PATCH /api/users, PATCH /api/users/:id/group; GET /api/security/groups, GET /api/security/privileges, GET /api/security/users/:id/effect access, PUT /api/security/groups/:id/privi GET/PATCH /api/work-orders; GET/POST/PATCH folha e tarefas em /api/work-orders/:id…; GET /api/geocode, GET /api/geocode-local; **GET/PUT /api/ref/types**; GET /api/teams; GET /api/mwm/demo-routes; GET /api/oms/work-orders/aggregate; GET /api/oms/work-orders; GET /api/audit/operations-archive; **GET /api/schematic/views**, **GET /api/schematic/views/:id**, **PATCH /api/schematic/views/:viewId/r GET /api/events**, **GET /api/events/export** (cursor after_id, opcional **shape=mongo** — ver docs/mds/EVENTS_ARCHITECTURE.m **GET /api/topology/meta**, **GET /api/topology/spans** (topologia PostGIS nativa — ver docs/mds/TOPOLOGY_NATIVE_POSTG Com **SECURITY_ENFORCE=1**: header obrigatório **X-TabGis-User-Id** e privilégios por rota (module.oms, module.mwm, module.aud,

Container	Imagem	Porta host	Função
infra-mongodb-1	mongo:latest	**27017**	Opcional / legado — trilha de eventos canónica em Postgres (events); ver docs/mds/EVENTS_ARCHITECTURE.m

Portas de acesso (de outro notebook na mesma rede ou via SSH tunnel)

Serviço	URL directa	Observação
Frontend (recomendado)	http://homeserver:8088/	nginx — same-origin para WMS, WFS, tiles e /api
GeoServer UI	http://homeserver:8080/geoserver/web	Admin directo
TileServer	http://homeserver:8081/	Acesso directo (diagnóstico)
PostGIS	homeserver:5433 (mapeamento em infra/docker-compose.yml)	Só admin / psql (porta 5432 é a interna à rede Docker)
MongoDB	homeserver:27017	Só admin

Credenciais de desenvolvimento (trocar em produção)

Serviço	Utilizador	Password
PostGIS	tabgis_admin	ver infra/docker-compose.yml
GeoServer	admin	geoserver (imagem padrão)

Dados e schemas

O banco de dados foi estruturado em *schemas* fundamentais para sustentar o comportamento da aplicação em ambiente agnóstico de idioma e localidade:

1. **landbase**: Contém todos os dados geográficos e topológicos puros do basemap e da rede (antigo basemap). Todas as camadas do GeoServer apontam para cá.
2. **config**: Estruturas de parametrização dinâmica (config.parameter), traduções globais (config.translation) e idiomas suportados (config.language: EN, PT, ES, IT).
3. **security**: Utilizadores em **security.users** (roles ADMIN, CONSULTA, OPERACAO); migração a partir de security.user em 01_init_schemas.sql.
4. **audit**: Histórico detalhado gerado por Triggers no PostGIS, gravando alterações em parâmetros (audit.parameter_history), sessões de usuário (audit.login_history), **arquivo obrigatório antes de DELETE** em tabelas operacionais (audit.operations_row_archive) e **audit.domain_event** (correlação de alterações — ver infra/sql/04_gis_oms_mwm_domain.sql).
5. **operations**: Equipas (operations.team), ordens (operations.work_order com owner_id e locked_by_id → security.users), deslocamentos equipa–ordem (operations.team_order_displacement), rotas por janela temporal (operations.team_route, ex. últimas 48h). Scripts: 03_operations_schema.sql, 04_gis_oms_mwm_domain.sql. Regra: *nada* é removido das tabelas principais de operations sem inserção prévia do registo em audit.operations_row_archive (triggers BEFORE DELETE).
6. **ref**, **gis**, **oms**, **mwm**: Modelo canónico utilities (consumidores, equipamentos, chamadas, ordens, equipas, rotas) conforme docs/mds/Database_Structure_Context.md; criado por 04_gis_oms_mwm_domain.sql.
7. **events**: Diário append-only (events.event) + função events.append_event (chamada **só** pelo sink na API Node; **sem triggers** em events) — ver docs/mds/EVENTS_ARCHITECTURE.md.
8. **schematic**: Vistas estáticas/sequenciais (schematic.network_view, schematic.view_node, schematic.view_edge); foco «Ir para o mapa» via focus_lat/focus_lng e/ou equipment_id → gis.equipment (infra/sql/16_schematic_schema opcional 17_schematic_link_poc_equipment.sql).

9. **Topologia nativa utilities**: extensão `postgis_topology` + schema da topologia `tabgis_utilities_topo` (primitivas `edge_data`, `node`, `face`, `relation`) e **segmentos lineares da rede** em `gis.network_span` / vista `gis.v_network_span_readable` — `infra/sql/21_postgis_topology_utilities.sql`; documentação `docs/mds/TOPOLOGY_NATIVE_POSTGIS.md`; API `GET /api/topology/meta`, `GET /api/topology/spans` (privilégio `module.oms` quando `SECURITY_ENFORCE=1`). Terminologia UI regionalizada: chaves `UI_GLOSSARY_*` / `UI_HELP_*` em `infra/sql/02_populate_translations.sql`.

Artefacto	Origem	Tabelas PostGIS	Camadas GeoServer
SP estado (activo)	IBGE UF + municípios; OSM PBF	`landbase.uf_sp`, `landbase.municipios_sp`, `landbase.vias_sp`	`base_sp:uf_sp`, `base_sp:municipios_sp`, `base_sp:vias_sp`
SP Centro (legado)	IBGE municípios; OSM Overpass	`landbase.municipio_sp`, `landbase.aoi_centro_sp`, `landbase.vias_centro_sp`	camadas homólogas

- **SRID administrativo (IBGE):** EPSG:4674 (SIRGAS 2000)
- **OSM (`osm2pgsql`):** EPSG:3857 na coluna `way`; `landbase.vias_sp` usa `ST_Transform(way, 4674)`
- **Vias por zoom:** tabela `landbase.via_zoom_config` + SLD `base_sp_vias_zoom_scales` — script de aplicação: `infra/scripts/apply_via_zoom_config_and_style.sh`

Recorte prioritário — municípios-alvo (RMSP ampliada)

24 municípios: São Paulo (capital), Barueri, Cajamar, Carapicuíba, Cotia, Diadema, Embu-Guaçu, Embu das Artes, Itapecerica da Serra, Itapevi, Jandira, Juquitiba, Mauá, Osasco, Pirapora do Bom Jesus, Ribeirão Pires, Rio Grande da Serra, Santana de Parnaíba, Santo André, São Bernardo do Campo, São Caetano do Sul, São Lourenço da Serra, Taboão da Serra, Vargem Grande Paulista.

Entregável	Tabela PostGIS	Camada GeoServer
Municípios-alvo destacados	`landbase.municipios_alvo_sp`	`base_sp:municipios_alvo_sp`
Bairros dos municípios-alvo	`landbase.bairros_alvo_sp`	`base_sp:bairros_alvo_sp`
Vias públicas dos municípios-alvo	`landbase.vias_alvo_sp`	`base_sp:vias_alvo_sp`

Script de geração: `infra/scripts/setup_municipios_alvo_sp.sh`

Scripts versionados

Script	Função	Runbook
`infra/sql/01_init_schemas.sql`	Schemas `landbase`, `config`, `security`, `audit` (tabelas base, parâmetros, histórico)	`docs/mds/DEPLOY.md` §2
`infra/sql/02_populate_translations.sql`	Chaves `config.translation` (EN/PT/ES/IT), `ON CONFLICT DO UPDATE`	`docs/mds/DEPLOY.md` §2
`infra/sql/03_operations_schema.sql`	Schema `operations` + arquivo `audit.operations_row_archive` + triggers `BEFORE DELETE`; `work_order.owner_id` / `locked_by_id` → `security.users`	`docs/mds/DEPLOY.md` §2

Script	Função	Runbook
infra/sql/04_gis_oms_mwm_domain.sql	Schemas ref, gis, oms, mwm; audit.domain_event; utilizadores e ordens iniciais	docs/mds/DEPLOY.md §2
infra/sql/05_operational_demo_data.sql	Dados fictícios operacionais (equipas com coordenadas, mais ordens e utilizadores)	docs/mds/DEPLOY.md §2
infra/sql/06_work_order_sheet_model.sql	Colunas decisórias; tabelas work_order_task / work_order_complaint; dados de exemplo; normalização de prefixos em order_code legado	docs/mds/DEPLOY.md §2
infra/sql/06_work_order_sheet_model_functions_triggers.sql	gen_random_work_order_code() (YYYYMMDD + sequência diária SP); triggers de código, contagem de reclamações e arquivo pré-delete	docs/mds/DEPLOY.md §2
infra/sql/07_work_order_oms_grid_columns.sql	Colunas OMS: started_at, callback_auto	docs/mds/DEPLOY.md §2
infra/sql/08_oms_test_orders_extra.sql	Dados de teste / vistas	aplicar manualmente conforme necessidade
…	OMS-MWM opcionais (ambiente de laboratório)	
infra/sql/13_oms_bulk_clustered_demo.sql	Massa demo densa	—
infra/sql/14_gis_equipment_network_demo.sql	Malha demo idempotente gis.equipment + gis.consumer (prefixos POC_DEMO_ / POC-DEMO-)	
infra/sql/15_events_schema.sql	Schema events + tabela events.event + events.append_event (diário multi-domínio)	docs/mds/EVENTS_ARCHITECTURE.md
infra/sql/16_schematic_schema.sql	Schema schematic (network_view, view_node, view_edge) + dados de demonstração de unifilar / P,Q	aba **ESQ** (Operação)
infra/sql/17_schematic_link_poc_equipment.sql	Liga nós da demo a gis.equipment POC (POC_DEMO_*)	opcional; enriquece **GET** /api/schematic/views/:id com tipo e location_desc
infra/sql/18_security_groups_privileges.sql	RBAC: security.privilege, security.user_group, security.group_privilege, security.user_group_member + security.users.is_active	Configurações → Utilizadores (desativação, grupo e privilégios)
infra/sql/19_fix_bairros_alvo_mapping.sql	Renomeia linhas em landbase.bairros_alvo_sp onde bairro_nome coincide com nome de município-alvo (ruído OSM)	Opcional pós-setup_municipios_alvo_sp.sh
infra/sql/20_sanity_bairros_alvo_sp.sql	Verificação: zero violações da mesma regra (falha psql se existirem)	CI / sanidade manual
infra/sql/21_postgis_topology_utilities.sql	Extensão postgis_topology; topologia tabgis_utilities_topo; gis.network_span (segmentos lineares) + TopoGeometry; dados demo TOPO_DEMO_SEG_*	docs/mds/TOPOLOGY_NATIVE_POSTGIS.md
infra/scripts/apply_poc_test_mass.sh	Executa o SQL 14 contra o contentor db (psql)	—

Script	Função	Runbook
`infra/scripts/repair_geosampa_mbtiles.sh`	Reparação incremental do MBTiles GeoSampa (só tiles em falta na tabela `tiles`)	`docs/mds/GeoSampa_offline_tecnico.md`
`infra/scripts/repair_geosampa_offline_tiles.sh`	Wrapper para a reparação; típico em background: `nohup … >> /tmp/geosampa_repair.log &`	`docs/mds/GeoSampa_offline_operacao.md`
`infra/scripts/setup_landbase_sp_estado.sh`	Landbase IBGE + OSM → PostGIS + GeoServer (SP estado)	`docs/mds/RUNBOOK_BASEMAP_SP_ESTADO.md`
`infra/scripts/setup_landbase_sp.sh`	Basemap São Paulo Centro (legado)	`docs/mds/RUNBOOK_BASEMAP_SP.md`
`infra/scripts/setup_municipios_alvo_sp.sh`	Municípios-alvo + bairros + vias	—
`infra/scripts/apply_via_zoom_config.sh`	Aplica SLD de vias por zoom	—
`infra/scripts/build_sp_satellite_mbtiles.sh`	Gera MBTiles raster (satélite offline)	ver secção abaixo

Estilos SLD versionados: `infra/geoserver/styles/*.sld`

MBTiles — satélite raster local

Estado actual do pacote

Campo	Valor
Ficheiro	`data/tiles/sp_satellite.mbtiles`
Tiles	**175 306**
Zoom	**10–17**
Bounds	`-47.2, -24.1, -46.2, -23.2` (capital SP)
Tamanho	~**3 GB**
Resolução	**1,2 m/px** (zoom 17)

Escala por ZMAX (bbox capital SP)

ZMAX	Tiles	Tempo est.	Disco est.	Resolução
14	2 959	~4 min	~50 MB	9 m/px
15	11 331	~25 min	~200 MB	5 m/px
16	44 271	~1h15	~763 MB	2,4 m/px
17	**175 306**	**~3,5 h**	**~3 GB**	**1,2 m/px** ← instalado
18	697 270	~14 h	~12 GB	0,6 m/px (impraticável neste recorte)

Procedimento de upgrade (aprovado: ZMAX=17)

```
# Gerar novo pacote (escreve em .tmp, substituição atómica no fim)
ZMAX=17 ./infra/scripts/build_sp_satellite_mbtiles.sh

# Após conclusão, reiniciar tileserver E web (nginx re-resolve DNS Docker)
docker compose -f infra/docker-compose.yml restart tileserver web
```

> **Nota Docker DNS:** o container `web` (nginx) resolve o hostname `tileserver` na rede interna Docker apenas no arranque. Se o `tileserver` for criado ou recriado depois do `web`, o nginx serve `index.html` para todos os pedidos `/tiles/*`. O restart conjunto resolve isso.

Comportamento da camada no frontend

- O toggle **Satélite local** faz fetch a `/tiles/data/sp_sat.json` antes de activar a camada.
- Se o TileServer responder → activa, mostra mensagem de satélite local activo (capital SP, zoom nativo 10–17).
- Se falhar → desactiva toggle, mostra mensagem com instrução de remediação.
- 404s de tiles fora da bbox são silenciados (cobertura parcial é comportamento normal).
- `minNativeZoom: 10` / `maxNativeZoom: 17` em `main.js` — Leaflet faz upscale acima e abaixo do intervalo.

Frontend

Ficheiro	Função
`frontend/index.html`	Topbar, Busca/Camadas, **Configurações** (modais: visualização, **utilizadores** com grupos/privilégios e desativação, **tipos** `ref.*`, traduções, **ajuda** `help.html`), painel **Operação** 50/50, folha OMS, toggles de camadas
`frontend/help.html`	Ajuda ao operador (resumo funcional; liga a `styles.css`)
`frontend/styles.css`	Tokens, topbar, painel operação, modais, clima (TMP), tabelas `ref` em modal
`frontend/main.js`	Leaflet, i18n, Operação, modais de configuração (inclui gestão de colaborador activo/inactivo, grupo e privilégios), **Open-Meteo** na guia TMP, `GET/PUT /api/ref/types`, vista inicial e zoom
`backend/server.js`	API Express — traduções, geocodificação, utilizadores, segurança RBAC, ordens/tarefas, `/api/ref/types`, equipas, auditoria, rotas demo MWM, `/api/schematic/*`, `/api/topology/*`, barramento `events` (`emitWithRequest`, `GET /api/events`, `GET /api/events/export`)
`backend/lib/*.test.js`	Testes unitários (`node:test`) — lógica pura OMS/geocode
`e2e/`	**Playwright** — testes de fumo (`specs/smoke.spec.ts`): mapa, traduções, menu Configurações
`scripts/`	Autonomia (`autonomy/`), stress HTTP (`stress/`) — ver `scripts/README.md`

- **Tipografia:** Inter (Google Fonts CDN); fallback `system-ui`
- **Modo recomendado (`:8088`):** GeoServer em `window.location.origin + '/geoserver'` (same-origin)
- **Modo dev (`:5173`):** GeoServer em `:8080`; WFS/GFI podem falhar por CORS

Topbar (uma única linha, altura `var(--topbar-height)` = 60px)

Zona	Conteúdo
Esquerda (`topbar-left`)	Marca TabGis, botão Início, botão **Operação** (abre/fecha split 50/50)
Centro (`topbar-titles-layer`, `pointer-events: none`, `z-index: 0`)	Camada absoluta sobre a barra: em tela **cheia** mostra só **Mapa** (tipografia próxima da marca, cor `var(--text-muted)`). Com **Operação aberta** (`.app.has-left-panel`): metade esquerda **Centro de Controle** (`UI_SPLIT_CONTROL_CENTER`), metade direita **Mapa** (`UI_SPLIT_MAP`) — alinhado às colunas 50/50 do conteúdo

Zona	Conteúdo
Direita (`topbar-right`)	**Atalho clima** (ícone cápsula sol+nuvem → guia TMP), **Busca** (WFS + geocodificação), **Camadas**, estado (`#status`), engrenagem (**idioma**, visualização, utilizadores, tipos `ref`, traduções, ajuda).

- **Busca vs filtro:** o rótulo do menu passou de «Filtros» para **«Busca»** (`UI_MENU_SEARCH`). A palavra *filtro* reserva-se a listagens tabulares futuras. Chaves de campo: `UI_SEARCH_MUNI_LABEL`, `UI_SEARCH_NEIGHBORHOOD_LABEL`, mensagens de *select* `UI_SEARCH_*`.

Painel Operação (split 50/50)

- **Classe** `.app.has-left-panel` no contentor raiz; `aside#left-panel` + `#map` dentro de `.split-main-row`.
- **Ordem das guias (tab bar):** 1) **OMS**, 2) **MWM**, 3) **DMS**, 4) **IND**, 5) **SMT**, 6) **ESQ** (esquemático / unifilar: `GET /api/schematic/views`, `GET /api/schematic/views/:id`, edição `PATCH .../nodes/:nodeId`; botão **Ir para o mapa** por nó usa foco do nó ou geometria de `gis.equipment` quando aplicável), 7) **TMP** (clima: cartões Open-Meteo + opcional **radar RainViewer** e **círculo de referência** no mapa Leaflet, controlos na própria guia), 8) **AUD**. A guia **Utilizadores** foi **retirada da barra**: a gestão de `security.users` passou para **Configurações → Gerenciar utilizadores** (modal). OMS consome `/api/work-orders` (e folha); utilizadores consomem `/api/users` a partir desse modal.
- **Cabeçalho do painel:** sem linha separada «Operação»; primeira linha = grelha de guias + botão fechar (`.operacao-top`).
- **Conteúdo:** tabelas e detalhe de ordens (OMS), equipas/rotas demo (MWM), política de auditoria (AUD); textos com `data-i18n` (prefixo `UI_OP_*`).
- **Tipos de domínio (`ref.*`):** não há guia no painel; edição de `label_en` / `label_pt` (códigos fixos) em **Configurações → Tipos de domínio (ref)** com **`GET/PUT /api/ref/types`** (ambiente dev sem autenticação por rota).
- **`SECURITY_ENFORCE=1`:** rotas OMS, MWM, AUD, ESQ e **topologia** (`/api/topology/*`, privilégio **`module.oms`**) e outras com `ensurePrivilege` devolvem **401/403** sem utilizador válido ou sem privilégio (ex.: **`module.esq`** para esquemático).

Vista inicial do mapa (`main.js`)

Constante	Valor	Significado
`HOME_INSET_FULL`	`0.9 × 1.04`	Tela cheia: bbox efectivo = 90% da meia-extensão × 1,04 (~4% menos «zoom agressivo» que 0,9 puro)
`HOME_INSET_SPLIT`	`HOME_INSET_FULL / 1.08`	Com painel 50%: ~8% mais zoom que a referência de tela cheia (compensa metade da largúra do mapa)
Bbox cheio	`-47.6480, -24.0674, -45.9180, -23.2733`	RMSP / capital
Bbox split	`-47.2496, -24.1088, -46.2513, -23.1966`	Recorte quando o mapa ocupa metade da largura

- **`setHomeView()`:** `map.invalidateSize()` + `requestAnimationFrame` → `fitBoundsHomeInset(...)` com o inset adequado. Botão **Inicial** e abertura do painel Operação usam a mesma função.
- **Arranque:** `map.whenReady` + `requestAnimationFrame` chama `runInitialMapLayout()` (define vista + `setStatus("Mapa carregado.")`). **Respaldo** `setTimeout(..., 400)` repete `setHomeView()` para contentores flex que só ganham altura após pintura; a mensagem de estado só é definida **uma vez** (`mapInitialStatusDone`).

Camadas alvo — zoom e toggles

- **Threshold:** zoom Leaflet **12** trata-se de «vista de detalhe».

- **Comportamento:** se a checkbox **Bairros** (`#bairros_alvo_sp`) está **marcada** em zoom 12, mostra-se `bairrosAlvoLayer` e remove-se `municipiosAlvoLayer` da vista (checkbox municípios-alvo desmarcada). Se **desmarcada**, mantêm-se municípios-alvo visíveis e bairros fora — útil para ver **rótulos de município** sem poluição dos bairros.
- **Zoom < 12:** bairros removidos, municípios-alvo repostos por defeito.

Estilos GeoServer (recorte alvo) — ficheiros em repo

SLD	Camada	Notas
`infra/geoserver/styles/base_sp_bairros_alvo.sld`		Preenchimento/contorno **suaves** (tons slate, opacidades baixas); rótulos `municipio_nome` + `bairro_nome` mais discretos
`infra/geoserver/styles/base_sp_municipios_alvo_highlight.sld`	Rótulos `nm_mun` **em todas as escalas** (removido `MinScaleDenominator` que escondia nomes em zoom próximo)	

Após alterar SLD no Git, republicar no GeoServer (Styles → substituir XML ou reexecutar script que aplica estilos, se existir no teu fluxo).

Chaves i18n relevantes (amostra)

Prefixo / chave	Uso
`UI_MENU_SEARCH`, `UI_SEARCH_*`	Menu Busca e textos dos selects
`UI_SPLIT_MAP`, `UI_SPLIT_CONTROL_CENTER`	Títulos na faixa central do topbar
`UI_BTN_OPERATION`	Botão Operação
`UI_OP_TAB_*`, `UI_OP_*`	Painel Operação (guias, colunas, mock, auditoria)

Lista completa mantida em `infra/sql/02_populate_translations.sql`; API devolve mapa chave → texto por idioma. **Glossário técnico (EN/PT/ES/IT):** `UI_GLOSSARY_LINEAR_NETWORK_SEGMENT`, `UI_GLOSSARY_POSTGIS_TOPOLOGY`, `UI_HELP_STACK_POSTGIS_TOPOLOGY`, `UI_HELP_API_TOPOLOGY` — termos sem duplo sentido coloquial (p.ex. «segmento linear da rede» em vez de «troço»).

Convenção de mensagens de erro

Todas as falhas em `main.js` seguem:

`"Falha em [recurso]. [Instrução de remediação] (ver docs/[RUNBOOK].md)."`

Ponto de falha	Mensagem
Tiles WMS	`Falha ao carregar tiles WMS. Verifique GeoServer em <URL> (ver docs/mds/DEPLOY.md).`
Tiles MBTiles	`Falha tiles MBTiles. Gere sp_satellite.mbtiles e suba tileserver (ver docs/mds/DEPLOY.md).`
WFS municípios	`Falha ao carregar municípios (WFS). Prefira http://HOST:8088 e verifique GeoServer (ver docs/mds/DEPLOY.md).`

Ponto de falha	Mensagem
Busca de bairros	`Falha na busca de bairros. Verifique GeoServer em <URL> (ver docs/mds/DEPLOY.md).`
GetFeatureInfo	`Falha ao consultar feição. Verifique GeoServer em <URL> (ver docs/mds/DEPLOY.md).`

Acesso remoto — outro notebook via Cursor SSH

Para continuar o desenvolvimento neste repositório a partir de outro notebook:

1. **Configurar SSH no notebook cliente** (`~/.ssh/config`):

   ```
   Host homeserver
       HostName <IP-do-homeserver>
       User wesley
       IdentityFile ~/.ssh/id_rsa   # ou a chave que já usa
   ```

2. **Abrir no Cursor:** `File → Connect to Remote → homeserver` (extensão Remote SSH)
3. **Abrir pasta:** `/home/wesley/projects/TabGis`
4. **Aceder ao frontend:** `http://homeserver:8088/` (ou `http://<IP>:8088/` se não tiver DNS local)
5. **Verificar stack antes de trabalhar:**

   ```
   docker ps --format "table {{.Names}}\t{{.Status}}"
   # Todos devem aparecer como "Up ... (healthy)" ou "Up ..."
   # Se algum estiver ausente:
   docker compose -f infra/docker-compose.yml up -d
   ```

   Cursor Rules: o ficheiro `.cursorrules` na raiz do repositório é lido automaticamente pelo Cursor em cada sessão — contém as regras de contexto e comportamento da IA para este projecto.

Definições técnicas (TabGis)

- O esquemático será **estático/sequencial** com metadado **"Ir para o Mapa"** por objecto.
- O versionamento utilizará lógica de instâncias **reais vs. históricas** (bi-temporal).
- Foco actual: landbase offline operacional; topologia de **segmentos lineares da rede** com `postgis_topology` (`gis.network_span`); evolução — edição REST, camadas mapa e CRS projetado onde necessário.

Próximos passos

Prioridade	Tarefa	Estado
1	Upgrade satélite MBTiles para ZMAX=17 (1,2 m/px)	**Operacional**
2	UI Operação + schema `operations` + arquivo `audit.operations_row_archive`	**Estrutura base + mock UI**

Prioridade	Tarefa	Estado
3	Modelagem explícita da topologia de rede (PostGIS Topology nativo)	**MVP:** `tabgis_utilities_topo` + `gis.network_span` + API leitura; pendente edição, WFS e topologia projectada opcional
4	APIs/backend para edição, ordens, rotas 48h e diário `events` (exportável a Mongo/externo)	**Parcial:** mutações emitem eventos; `GET /api/events`; `GET /api/events/export` com cursor + formato Mongo; sink/webhook dedicado fora da API continua opcional
5	Vista esquemática (schema `schematic`, REST, guia **ESQ**, «Ir para o mapa»)	**POC** (SVG escada + PATCH nó; endurecer com auth real além de `SECURITY_ENFORCE`)
6	Endurecimento de segurança (credenciais, TLS, sessão)	Fora do âmbito desta fase

---

Índice de documentação Markdown

Ficheiro	Conteúdo
`docs/README.md`	**Home** do índice de documentação; liga a `docs/mds/*.md`, `docs/html/`, `docs/pdfs/`
`docs/mds/ARCHITECTURE_BPMN.md`	Diagrama de colaboração (Mermaid): GitHub, PostGIS, GeoServer, GeoSampa, regionalização, Mongo pendente, deploy, linguagens
`docs/mds/REGIONALIZATION.md`	Tabela de idiomas e cadastro `config.translation`
`docs/mds/MAPS_AND_SOURCES.md`	Mapas offline, origens, baseline datado
`docs/mds/CONTEXT_STATE.md`	Este ficheiro — estado da stack, frontend, SQL, estilos
`docs/mds/Escopo_Projeto_TabGis.md`	Escopo de produto TabGis, arquitectura e roadmap
`docs/mds/Escopo_Projeto_GeoUtil.md`	Referência histórica de escopo académico
`docs/mds/DEPLOY.md`	Guia principal de deploy
`docs/mds/DEPLOY_POC.md`	Deploy Docker legado, ordem SQL alargada, diagnóstico
`docs/mds/EVENTS_ARCHITECTURE.md`	Schema `events`, barramento Node, exportação Mongo-compatível
`docs/mds/TOPOLOGY_NATIVE_POSTGIS.md`	Topologia nativa `postgis_topology`, segmentos lineares `gis.network_span`, API `/api/topology/*`; glossário i18n em `02_populate_translations.sql`
`.github/workflows/ci.yml`	CI: testes unitários do backend (`npm test`)
`.github/workflows/pages.yml`	GitHub Pages: `docs/mds/*.md` → `docs/html/*.html` + PDFs + artefacto `_site/` (hub + páginas + `help/`)
`docs/mds/RUNBOOK_BASEMAP_SP_ESTADO.md`	Pipeline landbase SP estado
`docs/mds/RUNBOOK_BASEMAP_SP.md`	Pipeline SP Centro (legado)
`frontend/README.md`	Cliente web, modos :8088 / :5173, camadas